BULLETIN DE SÉCURITÉ · TLP:CLEAR

PHP2Shell

CVE-2026-109618 · CVE-2026-109619 — note de correctif, plan de remédiation, recommandations AWS et outil d'audit

9,2 CVSS V4 (×2)	**Pré-auth** RCE SANS AUTHENTIFICATION	**21/10/2026** CORRECTIFS UPSTREAM	**8.2.35+** 8.3.36 · 8.4.27 · 8.5.12

Faits établis, hypothèses et recommandations sont distingués dans le texte. Les mitigations ne remplacent pas le correctif ; une mise à jour ne nettoie pas un serveur déjà compromis.

Version 1.0 · informations vérifiées le 22/10/2026 · auteur : Pepperhot · sources datées en fin de chaque partie

Sommaire

Partie I Note de correctif PHP2Shell

- A. Synthèse exécutive
- B. Analyse technique
- C. Matrice de compatibilité et de risque
- D. Procédure de remédiation (Debian / Ubuntu)
- E. Mesures compensatoires (temporaires, non équivalentes au correctif)
- F. Détection et réponse à incident
- G. Recommandations AWS (synthèse)
- H. Plan de validation (checklist)
- I. Stratégie de maintenance
- Sources (consultées le 22/10/2026)

Partie II Plan de remédiation Debian / Ubuntu

- Étape 0 — Préparation (lecture seule)
- Étape 1 — Inventaire des versions installées
- Étape 2 — Version réellement utilisée par chaque site
- Étape 3 — Sauvegarde et compatibilité (modifie le système : go/no-go)
- Étape 4 — Mise à jour depuis une source confirmée (go/no-go)
- Étape 5 — Redémarrage contrôlé (go/no-go)
- Étape 6 — Vérification (lecture seule)
- Étape 7 — Contrôle post-déploiement et retour arrière
- Cas particuliers

Partie III Recommandations AWS

- 1. Partage des responsabilités
- 2. EC2 — mise à jour du système et du moteur PHP
- 3. AWS Systems Manager — inventaire et correctifs
- 4. AWS WAF (ALB, CloudFront, API Gateway, App Runner…)
- 5. Journalisation et investigation
- 6. Détection gérée : GuardDuty, Inspector, Security Hub
- 7. Conteneurs (ECS, EKS, ECR)
- 8. Lambda et environnements PHP personnalisés
- 9. IAM, secrets, réseau — réduction du rayon d'impact
- Sources (consultées le 22/10/2026)

Partie IV Outil php-vuln-audit : utilisation et maintenance

- Principes
- Installation
- Utilisation
- Contrôles
- Architecture
- Configuration
- Maintenance
- Limites connues

PARTIE I

Note de correctif PHP2Shell

Source : NOTE_CORRECTIF_PHP2SHELL.md

Champ	Valeur
Référence	PHP2SHELL-2026-10
Version de la note	1.0 — 22/10/2026
Informations vérifiées le	22/10/2026 (sources datées en fin de document)
Logiciel	Moteur PHP (php-src), tous SAPI servant des requêtes web
Gravité éditeur	Critique — CVSS v4.0 9,2 pour chaque CVE [S2][S3]
Correctifs upstream	8.5.12, 8.4.27, 8.3.36, 8.2.35 (21/10/2026) [S1]
Statut d'exploitation	Exploitation observée par watchTowr sur ses honeypots dès le 21/10 [S14] ; non confirmée par CERT-EU, absente du catalogue CISA KEV à la date de vérification [S6][S15]

Légende : **[Fait]** = confirmé par une source primaire citée ; **[Hypothèse]** = affirmation non confirmée par une source primaire ; **[Reco]** = recommandation de l'auteur de la note ; **À confirmer** = information non vérifiable à la date de rédaction.

A. Synthèse exécutive

Ce qu'est PHP2Shell. Nom d'usage (non officiel) de deux vulnérabilités mémoire du moteur PHP lui-même, publiées le 21/10/2026 [S1] :

- **CVE-2026-109618** — corruption mémoire lors de la gestion d'un *out-of-memory* (OOM) : PHP interrompt la requête par `longjmp()` alors que des données de requête sont partiellement initialisées ; le nettoyage de fin de requête les traite ensuite comme valides [S2]. **[Fait]**
- **CVE-2026-109619** — état global périmé : un état appartenant à une requête survit à la libération de sa mémoire ; une requête ultérieure traitée **par le même worker** peut l'utiliser avant réinitialisation (use-after-free) [S3]. **[Fait]**

Les deux peuvent permettre une **exécution de code à distance sans authentification**, avec les privilèges du compte du worker web (`www-data` sous Debian/Ubuntu) [S1][S2][S3]. **[Fait]**

Conséquences métier. Prise de contrôle du serveur web au niveau du compte applicatif : lecture des fichiers de configuration (identifiants de base de données, clés API), accès aux données traitées par l'application, dépôt de portes dérobées, rebond vers le réseau interne et, sur AWS, utilisation des identifiants du rôle d'instance. Le défaut est dans l'interpréteur : **mettre à jour WordPress, Drupal, Joomla ou tout autre CMS ne corrige rien** [S16].

Conditions d'exploitation connues [Fait, S2][S3].

- Aucune authentification, aucune interaction utilisateur. Complexité d'attaque élevée (AC:H) et prérequis d'attaque présents (AT:P) dans le vecteur CVSS.
- **Apache mod_php** : exposition la plus forte ; RCE validée par HTTP avec limites mémoire et d'entrée par défaut, déclenchement **avant tout code applicatif** [S1][S2].
- **PHP-FPM** : CVE-2026-109619 validée via le chemin *session upload progress* ; CVE-2026-109618 « susceptible » d'après l'analyse du code, exploit complet non démontré [S2][S3].

- FrankenPHP, LiteSpeed, php-cgi : concernés selon l'éditeur ; niveau de démonstration variable [S1].
- La fiabilité dépend du build, des extensions, du frontal et de l'agencement de l'allocateur [S2]. Selon VulnCheck (cité par Tanium, source secondaire), aucune exploitation publique ne contourne l'ASLR sur x86-64 à ce jour [S17] — **À confirmer** ; un dépôt public prétend le contraire en laboratoire (voir §B.6).

Versions concernées [Fait, S1][S4][S5]. PHP 8.2 < 8.2.35, 8.3 < 8.3.36, 8.4 < 8.4.27, 8.5 < 8.5.12, ainsi que **toutes** les versions 5.x, 7.x, 8.0.x et 8.1.x (non maintenues, sans correctif upstream).

Priorité : P1 — traitement immédiat pour tout serveur exposé à Internet exécutant PHP en mod_php ou FPM.

Mesures immédiates [Reco].

1. Inventorier les versions réellement exécutées (paquets **et** binaires hors paquets) — `python3 phpaudit.py audit`.
2. Installer le paquet corrigé de la distribution (ou la version upstream corrigée) puis **redémarrer complètement** PHP-FPM / Apache.
3. Si la correction est impossible sous quelques heures : isoler le serveur des réseaux publics (recommandation CERT-EU et CERT-FR [S4][S6]) ; à défaut, appliquer les mitigations du §E en sachant qu'elles **ne rendent pas l'installation sûre** [S1].
4. Rechercher des signes de compromission (§F), en particulier sur les serveurs restés exposés depuis le 20/10/2026.

B. Analyse technique

B.1 Fonctionnement normal d'une requête PHP

1. **Démarrage de requête** : le SAPI (mod_php, FPM…) reçoit la requête, applique la configuration (php.ini, pool FPM, directives Apache, `.htaccess` sous mod_php), puis PHP analyse le corps (`application/x-www-form-urlencoded`, `multipart/form-data` selon RFC 1867) pour remplir `$_POST` et `$_FILES`. Cette phase s'exécute **avant** le script.
2. **Exécution** du script ; toutes les allocations « par requête » passent par le gestionnaire mémoire de Zend, borné par `memory_limit`.
3. **Fin de requête** : destruction des objets, fermeture de session, libération en bloc de la mémoire de requête.
4. **Réutilisation du worker** : un worker FPM ou un enfant Apache prefork traite des centaines de requêtes successives. Certains états globaux persistent volontairement entre requêtes ; ils doivent être réinitialisés **avant** que la mémoire qu'ils référencent soit libérée.

B.2 CVE-2026-109618 — corruption lors d'un OOM [Fait, S2]

- PHP gère l'épuisement mémoire par `longjmp()` pour abandonner la requête. Si l'allocation échoue alors que des structures de requête sont partiellement initialisées, la sortie anticipée n'est pas prise en compte : le nettoyage de fin de requête lit ou exploite un état invalide. L'allocation qui échoue n'a pas besoin d'être liée à la mémoire invalide.
- Stratégies OOM décrites par l'éditeur dans le parsing multipart : requêtes **sans `Content-Length` exploitable** (contournant la vérification initiale de `post_max_size`), parties multipart avec un nombre illimité d'en-têtes (forte amplification mémoire interne), contrôles de fichier vides et noms de champs imbriqués gonflant les tableaux ; d'autres chemins applicatifs peuvent déclencher l'OOM.
- **Sessions : non mentionnées** dans l'avis de cette CVE.
- Correctif : remplacement du chemin OOM par `longjmp()` par un OOM « doux » différé au prochain point sûr de la VM, plus un OOM « dur » immédiat au-delà d'un seuil supérieur.

B.3 CVE-2026-109619 — état global périmé [Fait, S3]

- Un **producteur** (requête qui publie un état appartenant à la requête dans une globale) et un **consommateur** (requête ultérieure sur le même worker qui lit cet état avant réinitialisation) sont nécessaires.
- Deux façons de laisser l'état : erreurs fatales (OOM, erreurs de parsing/compilation, `require` en échec, *Maximum execution time exceeded*…) qui interrompent avant la remise à zéro ; diagnostics non fatals émis pendant l'arrêt qui réécrivent un état d'erreur déjà nettoyé.
- Chemins identifiés :
- **session.upload_progress** — activé par défaut, ne nécessite pas `session_start()`, atteint la gestion de session avant l'initialisation normale ; validé contre mod_php et FPM.
- **fatal_error_backtraces (PHP 8.5)** — cas le plus solide, base de la RCE sur mod_php.
- **Diagnostics INI avant initialisation** — avertissements émis lors de l'application de directives par requête (obsolètes ou invalides), indépendamment de `error_reporting`, `@`, `display_errors`, `log_errors`. Sous mod_php, `.htaccess` et `php_value` s'appliquent tôt ; FPM est exposé de façon conditionnelle.
- Autres états (erreurs, exceptions, objets, extensions) : conditionnels, dépendent de l'application.
- **FPM** : producteur et consommateur doivent atteindre le même worker ; plusieurs workers et le recyclage réduisent la probabilité sans supprimer la condition. FPM tue un enfant dont le démarrage échoue.

B.4 Différences entre les deux CVE

Critère	CVE-2026-109618	CVE-2026-109619
Nature	Corruption pendant le nettoyage après OOM	Use-after-free d'état global entre deux requêtes
Déclencheur	Échec d'allocation (OOM) en cours de requête	Fatal/diagnostic laissant un état, puis consommation par une requête suivante
Requêtes nécessaires	Une requête peut suffire (mod_php)	Producteur + consommateur sur le même worker
Rôle de `session.upload_progress`	Aucun selon l'avis	Chemin confirmé (mod_php et FPM)
Mitigation partielle principale	`memory_limit=-1` + limites de taille/structure en amont	`session.upload_progress.enabled=0` ; `fatal_error_backtraces=0` (8.5)
SAPI démontré	mod_php (RCE validée)	mod_php (RCE), FPM (chemin session validé)

B.5 Rôle des composants cités

Composant	Rôle confirmé	Source
Uploads HTTP (multipart)	Vecteur principal de l'OOM (618) et du chemin session (619)	[S2] [S3]
Sessions	Chemin *upload progress* de 619 ; absent de 618	[S3]
Erreurs mémoire	Déclencheur de 618 ; producteur possible de 619	[S2] [S3]
OPcache	**Non mentionné** dans les avis GHSA. L'affirmation du document initial liant 619 à OPcache n'est pas étayée : **[Hypothèse non confirmée]**	[S2] [S3]
PHP-FPM	Concerné ; réutilisation de worker = condition de 619 ; `pm.max_requests=1` supprime cette réutilisation	[S3]
PHP CLI	Non mentionné comme surface d'attaque (pas de parsing de requête HTTP). Le binaire CLI partage le code du paquet : à mettre à jour, priorité moindre	[S1]

B.6 Scénarios d'impact

- **RCE** : exécution de code natif dans le worker. Conséquence importante : `disable_functions`, `open_basedir` et les autres restrictions de niveau PHP **ne protègent pas** contre du code natif injecté. [Reco, déduit de la nature de la faille]
- **Fuite de données** : divulgation mémoire explicitement citée pour 619 [S3] (fragments d'autres requêtes : cookies, en-têtes, données de formulaires) ; ensuite tout ce que le compte du worker peut lire.
- **Compromission applicative** : modification de fichiers PHP inscriptibles, comptes administrateurs, contenus.
- **Rebond** : base de données et services joignables depuis le serveur, métadonnées cloud (IMDS) et identifiants de rôle, élévation de privilèges locale si le noyau ou la configuration le permettent.

B.7 Prérequis et limites de l'exploitation

- Complexité élevée ; résultats variables selon architecture, build et allocateur [S2]. Red Hat juge la RCE difficile en pratique sur FPM par défaut (rapporté par Tanium, source secondaire) [S17] — **À confirmer** sur la source Red Hat.
- Un worker exploité peut planter : crashs et 500 sont des symptômes possibles, pas une preuve.
- Les mitigations par configuration ne couvrent que des chemins **confirmés** ; d'autres variantes existent [S1] [S3].

B.8 Analyse des dépôts GitHub fournis

Dépôt	Nature	Points concordants	Écarts / alertes
NCSC-NL/php2shell	Référentiel opérationnel d'un CSIRT national (TLP:CLEAR, en cours d'enrichissement)	Descriptions, versions corrigées, SAPI, mitigations alignés sur php.net	Pas d'IOC ni de signature à la date de lecture ; répertoire `detection_mitigation` non consultable (robots) — **À confirmer**
ikow/php2shell-poc	Laboratoire Docker + analyse (FPM/nginx 8.2.34, build ASAN)	Versions corrigées identiques	Attribue les **deux** CVE à `session.upload_progress` (faux pour 618) ; chaînes nécessitant LFI ou `unserialize()` applicatifs, contrairement à la RCE pré-applicative décrite pour mod_php ; recommande `session.use_only_cookies` (absent des avis) ; résultats auto-déclarés
9bie/php2shell-bypass-aslr-exp	Write-up de recherche, mod_php prefork, PHP 8.4.26 compilé sans patch, Debian 12	Cohérent avec l'exposition mod_php	Aucun identifiant CVE ; `memory_limit=8M` de laboratoire ; contournement ASLR revendiqué sur un seul build, petits échantillons ; attribué à un modèle d'IA ; non reproductible en l'état
syrex1013/php2shell	**Outil offensif** (dépôt de webshell, empoisonnement de logs) ; 2 commits, 0 étoile	Versions corrigées identiques	Décrit les deux CVE comme un UAF de `session.upload_progress` (contredit 618) ; prérequis LFI/port FastCGI exposé/pearcmd ; **ne jamais l'exécuter hors laboratoire isolé** ; des faux PoC distribuent des malwares [S17]

Usage défensif retenu : les indicateurs de chaînage (LFI vers `pearcmd`, PHP dans les répertoires d'upload, port FastCGI exposé) alimentent les règles DR-011/DR-012 et le §E réseau. Aucun payload n'est repris.

B.9 Corrections apportées au document initial

Affirmation initiale	Statut
Correctifs pour 8.2 et supérieures le 21/10/2026	**Confirmé** [S1]

Affirmation initiale	Statut
CVSS 4.0 9,2	**Confirmé** [S2][S3]
618 implique « uploads, sessions »	**Partiellement faux** : uploads oui, sessions non (les sessions relèvent de 619) [S2]
619 implique OPcache	**Non étayé** par les avis [S3]
`session.upload_progress.enabled=Off` = mesure recommandée	**Incomplet** : ne couvre que les chemins session de 619, rien pour 618 [S1][S4]
« Wiz a démontré sur WordPress, Drupal et Joomla »	**Non confirmé**. Un chercheur Wiz figure parmi les rapporteurs [S2] ; la publication Wiz « wp2shell » concerne d'autres CVE (CVE-2026-63030/60137) [S18]
ENISA confirme	**À confirmer** (référence ENISA CNW-2026-04 citée par NCSC-NL, non consultée directement)
`sudo apt upgrade`	**Insuffisant comme garantie** : voir §D (vérification de la version candidate et des backports)
Surveiller les PHPSESSID volumineux	**Hypothèse** issue d'un dépôt de recherche, absente des avis officiels

C. Matrice de compatibilité et de risque

C.1 Branches upstream (état au 22/10/2026)

Branche	Maintenance [S12]	Vulnérable	Version corrigée	Source	Action
8.5	Active, sécurité jusqu'au 31/12/2029	< 8.5.12	**8.5.12**	[S1]	Mettre à jour ; en attente : `fatal_error_backtraces=0`
8.4	Active, sécurité jusqu'au 31/12/2028	< 8.4.27	**8.4.27**	[S1]	Mettre à jour
8.3	Sécurité seule, jusqu'au 31/12/2027	< 8.3.36	**8.3.36**	[S1]	Mettre à jour
8.2	Sécurité seule, **fin le 31/12/2026**	< 8.2.35	**8.2.35**	[S1]	Mettre à jour + planifier migration 8.4/8.5
8.1, 8.0, 7.4	Fin de vie	Toutes	Aucune upstream ; backports source minimaux publiés par l'équipe PHP	[S1]	Backport fournisseur vérifié ou migration ; isoler en attendant
≤ 7.3, 5.x	Fin de vie	Toutes [S4] [S5]	Aucune upstream	[S4]	Migration ; backport commercial uniquement si vérifié

C.2 Paquets de distribution (rétroportages : le numéro upstream ne change pas)

Distribution	Paquet source	Version corrigée	Avis	Source
Debian 13 trixie	php8.4	8.4.27-1~deb13u1	DSA-6574-1	[S7]
Debian 12 bookworm	php8.2	8.2.35-1~deb12u1	DLA-4848-1	[S7] — la fiche 619 indiquait encore « vulnerable » le 22/10 alors que 8.2.35 corrige les deux CVE : **À confirmer**

Distribution	Paquet source	Version corrigée	Avis	Source
Debian 11 bullseye	php7.4	Hors Debian LTS (fin 31/08/2026) ; ELTS Freexian payant	—	Source secondaire [S19] — **À confirmer**
Debian forky/sid	php8.4	Non corrigé au 22/10 (8.4.26-1)	—	[S7]
Ubuntu 26.10	php8.5	8.5.9-0ubuntu2.1	USN-8948-1	[S8]
Ubuntu 26.04 LTS	php8.5	8.5.4-0ubuntu1.5	USN-8948-1	[S8]
Ubuntu 24.04 LTS	php8.3	8.3.6-0ubuntu0.24.04.13	USN-8948-1	[S8]
Ubuntu 22.04 LTS	php8.1	8.1.2-1ubuntu2.28	USN-8948-1	[S8]
Ubuntu 20.04 / 18.04	php7.4 / php7.2	7.4.3-4ubuntu2.29+esm5 / 7.2.24-0ubuntu0.18.04.17+esm14	USN-8948-2 (Ubuntu Pro)	[S9]
Ubuntu 16.04 / 14.04	php7.0 / php5	7.0.33-0ubuntu0.16.04.16+esm22 / 5.5.9+dfsg-1ubuntu4.29+esm18	USN-8948-2 (Legacy)	[S9]
Dépôt Sury (deb.sury.org, PPA ondrej)	php8.2–8.5	Versions upstream 8.2.35 / 8.3.36 / 8.4.27 / 8.5.12	—	[S20] (secondaire) ; builds Launchpad 8.4.27 visibles [S21]
Dépôt Sury	php5.6–8.1	Backports annoncés, chaînes exactes non publiées	—	**À confirmer** via changelog du paquet
Remi (RPM)	—	Pas de mise à jour le 21/10 ; l'équipe PHP recommande paquets distribution ou compilation	—	[S1]

C.3 Par mode d'exécution

Mode	Exposition	Spécificités de correction	Vérification
PHP CLI	Non mentionné comme vecteur	Même paquet source ; redémarrer les workers CLI longue durée (queues, supervisord, Laravel `queue:restart`)	`dpkg-query` + contrôle `/proc/*/maps`
PHP-FPM	619 validée (session) ; 618 susceptible	Paquet `phpX.Y-fpm` + `systemctl restart phpX.Y-fpm`	Versions, absence de mappings `(deleted)`, pools actifs
Apache mod_php	**Maximale** (618 et 619 confirmées, avant code applicatif)	Paquet `libapache2-mod-phpX.Y` + **restart** Apache (pas `reload` / `graceful`) [S19]	`apache2ctl -M`, mappings `libphp*.so (deleted)`
Conteneurs	Celle du SAPI embarqué	Reconstruire depuis une image corrigée (`--pull`), redéployer ; une image corrigée dans le registre ne protège pas un conteneur en cours [S22]	`php -v` dans le conteneur, SBOM, digest

Mode	Exposition	Spécificités de correction	Vérification
Compilé / embarqué (`/usr/local` , `/opt/bitnami` , FrankenPHP, appliances)	Celle du SAPI	`apt` ne met rien à jour : recompiler depuis 8.x corrigée, mettre à jour la pile fournisseur ; branches 7.4–8.1 : backports source [S1][S11]	Contrôle `BIN` de l'outil (binaire non possédé par dpkg)

D. Procédure de remédiation (Debian / Ubuntu)

La procédure complète, commande par commande, avec objectif, résultat attendu, critères de go/no-go et retour arrière, est dans **`PLAN_REMEDIATION.md`** . Principes :

1. **Inventaire** : paquets (`dpkg-query`), binaires hors paquets, services, pools, modules Apache — automatisé par `phpaudit.py collect` .
2. **Version réellement servie par chaque site** : correspondance vhost → handler (`SetHandler proxy:unix:…` , `fastcgi_pass`) → pool → branche ; `php -v` ne renseigne que le CLI.
3. **Sauvegarde** de `/etc/php` , `/etc/apache2` , `/etc/nginx` , de la liste des paquets et, idéalement, snapshot machine.
4. **Mise à jour depuis une source fiable** : vérifier **avant installation** que la version *candidate* (`apt-cache policy`) est ≥ à la version corrigée de la matrice C.2 ; mettre à jour uniquement les paquets PHP (`apt-get install --only-upgrade …`).
5. **Redémarrage complet** des services concernés.
6. **Vérification** : version installée, CVE présentes dans `changelog.Debian.gz` , aucun processus avec bibliothèque PHP supprimée, tests fonctionnels.
7. **Surveillance et retour arrière** (le retour arrière réintroduit la faille : uniquement sur serveur isolé).

Rétroportages. Debian et Ubuntu corrigent sans changer la version upstream : Ubuntu 24.04 corrigé affiche toujours `PHP 8.3.6` dans `php -v` . La preuve est la **version complète du paquet** comparée à l'avis (USN/DSA/DLA) et la présence des identifiants CVE dans le changelog :

```
dpkg-query -W -f='${Version}\n' php8.3-common          # attendu (24.04) : >= 8.3.6-0ubuntu0.24.04.13
zgrep -E 'CVE-2026-10961[89]' /usr/share/doc/php8.3-common/changelog.Debian.gz   # attendu : les 2 CVE
dpkg --compare-versions "$(dpkg-query -W -f='${Version}' php8.3-common)" ge 8.3.6-0ubuntu0.24.04.13 && echo CORRIGÉ
```

Le script `remediation/planner.py` génère ces contrôles automatiquement pour l'hôte audité et bloque l'installation si la version candidate n'est pas confirmée.

E. Mesures compensatoires (temporaires, non équivalentes au correctif)

> L'équipe PHP : « There are no complete mitigations, updating is the only safe choice » [S1]. Toute directive ci-dessous doit être posée là où elle s'applique **avant** le parsing de la requête (php.ini, pool FPM, configuration Apache), puis **tous les workers redémarrés**. `ini_set()` et `.user.ini` sont inopérants [S1] [S3].

E.1 `session.upload_progress.enabled = Off` (CVE-2026-109619)

Syntaxe vérifiée	php.ini : `session.upload_progress.enabled = Off` ; pool FPM : `php_admin_flag[session.upload_progress.enabled] = off` [S5] ; Apache/mod_php (conf serveur ou vhost) : `php_admin_flag session.upload_progress.enabled off`
Périmètre	Bloque les chemins de session confirmés de 619 uniquement ; **aucun effet sur 618** [S1][S17]
Effets de bord	Désactive la progression d'upload côté serveur (`$_SESSION[...]` de progression) ; les barres de progression JavaScript basées sur XHR ne sont pas concernées
Tests	`php-fpmX.Y -t` ; valeur effective par pool via `phpaudit.py audit --checks MIT` ; test d'upload fonctionnel
Risque résiduel	Autres producteurs/consommateurs d'état (fatals, diagnostics INI)
Point d'attention	Sous mod_php, un `.htaccess` (`php_value` / `php_flag`) peut réactiver la directive : n'utiliser que `php_admin_flag` . L'avis recommande de la laisser désactivée **même après correctif** si inutilisée [S3]

E.2 `fatal_error_backtraces = 0` (PHP 8.5 uniquement, CVE-2026-109619)

Bloque le seul chemin *fatal backtrace* confirmé (base de la RCE mod_php) [S3]. Effet de bord : traces de pile absentes des erreurs fatales (diagnostic plus difficile). Pool FPM : `php_admin_value[fatal_error_backtraces] = 0` .

E.3 `memory_limit = -1` + limite cgroup (CVE-2026-109618)

Syntaxe	Pool FPM : `php_admin_value[memory_limit] = -1` ; Apache : `php_admin_value memory_limit -1` ; vérifier qu'aucun pool, répertoire ou application ne réapplique une limite finie [S1]
Périmètre	Supprime le déclenchement de l'OOM interne PHP (cœur de 618)
Effets de bord	**Risque de déni de service** : une requête peut consommer toute la mémoire de l'hôte ; certaines applications ajustent `memory_limit` par `ini_set()`
Compensation	Limite mémoire cgroup/conteneur, « de préférence avec une politique qui tue seulement le worker concerné » [S1]. Exemple systemd **[Reco — à valider en préproduction]** : drop-in `MemoryMax= + OOMPolicy=continue` sur `phpX.Y-fpm.service` (le master FPM relance l'enfant tué). **Ne pas utiliser `RLIMIT_AS / RLIMIT_DATA`** : elles peuvent provoquer l'échec d'allocation vulnérable [S1][S2]
Tests	Charge réaliste en préproduction ; surveiller `MemoryCurrent` (`systemctl show phpX.Y-fpm -p MemoryCurrent`)
Risque résiduel	OOM déclenchables par d'autres voies (allocations système, extensions)

E.4 Limites de taille et de structure des requêtes (CVE-2026-109618)

Mesure	Syntaxe	Remarque
`post_max_size`	`8M` sans upload, `32M` avec upload ; `upload_max_filesize` < `post_max_size` [S1]	Insuffisant seul : requêtes sans `Content-Length` exploitable
Nombre de parties / variables / imbrication / fichiers	`max_multipart_body_parts` , `max_input_vars` , `max_input_nesting_level` , `max_file_uploads`	Réduire aux besoins réels de l'application [S2]
Nginx	`client_max_body_size 32m;`	Comportement exact vis-à-vis des corps *chunked* et de `CONTENT_LENGTH` transmis à FastCGI : **À confirmer** en préproduction

Mesure	Syntaxe	Remarque
Apache	`LimitRequestBody 33554432`, `LimitRequestFields 50`, `LimitRequestFieldSize 8190`	Application aux corps *chunked* : **à vérifier** en préproduction
Nombre d'en-têtes par partie multipart	Aucun réglage PHP connu	Uniquement WAF/proxy capable d'inspecter le multipart

E.5 WAF

- CERT-FR : bloquer les requêtes `Content-Type: multipart/*` ou, à défaut, les superviser [S5]. **Effet de bord : casse tous les formulaires d'upload.** Variante ciblée [Reco] : bloquer le multipart sur les chemins qui n'acceptent jamais d'upload.
- Règles AWS (custom `PHP2Shell` + `SizeRestrictions_BODY` + groupe PHP v2.3) : voir `AWS_SECURITY.md`. Limites démontrées : inspection limitée aux 8 premiers Ko du corps sur ALB ; le PoC public place les marqueurs vers 56 Ko [S13][S23].
- Toute règle par signature (`PHP_SESSION_UPLOAD_PROGRESS`, `PHPSESSID`) est contournable si l'attaque cible 618 ou si `session.name` / `session.upload_progress.name` diffèrent ; démarrer en mode *Count*.

E.6 Réseau, moindre privilège, isolation [Reco]

- **Isolement** des serveurs non corrigeables (CERT-EU, CERT-FR) [S4][S6].
- Ne jamais exposer FastCGI : `ss -ltnp | grep -E ':900[0-9]'` doit être vide côté interfaces publiques (prérequis de certains PoC [B.8]).
- Filtrage sortant des serveurs web (limite le téléchargement d'outils et les shells inversés).
- Un pool FPM par site avec utilisateur dédié ; webroot **non inscriptible** par l'utilisateur du pool hors répertoires d'upload ; interdire l'exécution PHP dans les répertoires d'upload.
- Durcissement systemd de `phpX.Y-fpm` (`NoNewPrivileges=yes`, `PrivateTmp=yes`, `ProtectSystem=strict` + `ReadWritePaths=`, `ProtectHome=yes`) : à tester en préproduction.
- `pm.max_requests = 1` supprime la réutilisation de worker exigée par 619, au prix d'un coût CPU important [S3].
- Préférer PHP-FPM à mod_php (isolation par pool, exposition moindre).

F. Détection et réponse à incident

Toutes les commandes sont **en lecture seule**. L'outil (`phpaudit.py audit`, contrôle `LOG`) automatise les recherches de journaux via `config/detection_rules.json`.

F.1 Journaux et requêtes

```
# POST terminés en 500, par IP (format combined : $6="POST, $9=statut)
zcat -f /var/log/apache2/access.log* /var/log/nginx/access.log* 2>/dev/null | awk '$6=="\"POST" && $9==500
{print $1}' | sort | uniq -c | sort -rn | head -20
# Épuisement mémoire PHP
zgrep -hE 'Allowed memory size of [0-9]+ bytes exhausted|Out of memory \(allocated' /var/log/apache2/error.l
og* /var/log/nginx/error.log* /var/log/php*-fpm.log* 2>/dev/null | tail -50
# Workers tués par signal
zgrep -hE 'exited on signal (6|7|11)' /var/log/php*-fpm.log* 2>/dev/null | tail -50
zgrep -h 'AH00052' /var/log/apache2/error.log* 2>/dev/null | tail -50
# Segfaults noyau et dumps
journalctl -k --since 2026-10-15 --no-pager | grep -E 'segfault|general protection'
coredumpctl list --no-pager 2>/dev/null | grep -Ei 'php|apache'
# Redémarrages de services
journalctl -u 'php*-fpm*' -u apache2 -u nginx --since 2026-10-15 -o short-iso --no-pager | grep -Ei 'start|s
top|fail|kill'
# Diagnostics INI pré-initialisation (consommateur 619)
zgrep -hE 'PHP (Warning|Deprecated|Notice):.* in Unknown on line 0' /var/log/apache2/error.log* /var/log/php
*-fpm.log* 2>/dev/null | sort | uniq -c | sort -rn | head
```

Les formats de log par défaut n'enregistrent ni `Content-Type`, ni taille de requête, ni corps. Pour la suite (à activer après validation) :

- Apache (mod_logio pour `%I`) : `LogFormat "%h %l %u %t \"%r\" %>s %b %I \"%{Content-Type}i\" %D" php2shell`
- Nginx : `log_format php2shell '$remote_addr - $remote_user [$time_local] "$request" $status $body_bytes_sent $request_length "$content_type" $request_time';`

F.2 Processus et réseau

```
# Processus du compte web qui ne sont pas des workers (shell, curl, wget, python, nc…)
ps -u www-data -o pid,ppid,lstart,cmd --no-headers | grep -vE 'php-fpm|apache2|nginx'
ps -eo pid,ppid,user,lstart,cmd --forest | grep -B1 -A3 -E 'php-fpm: pool|apache2 -k'
# Connexions sortantes établies par les workers
ss -tnp state established | grep -E 'php-fpm|apache2'
# Processus exécutant encore l'ancien PHP
grep -lE '(php|libphp|apache2).*\(deleted\)$' /proc/[0-9]*/maps 2>/dev/null
```

F.3 Fichiers et persistance

```
# PHP créés/modifiés depuis l'annonce
find /var/www -xdev -type f \( -name '*.php' -o -name '*.phtml' -o -name '*.phar' -o -name '.htaccess' -o -n
ame '.user.ini' \) -newermt '2026-10-20' -printf '%TY-%Tm-%Td %TT %u %m %p\n' | sort
# PHP dans les répertoires d'upload / temporaires
find /var/www -xdev -ipath '*upload*' -name '*.ph*' -ls
find /tmp /var/tmp /dev/shm -xdev -type f -newermt '2026-10-20' -ls
# Fichiers de session anormalement volumineux
find /var/lib/php/sessions -type f -size +100k -ls 2>/dev/null
# Injection par prepend/append
grep -RnE 'auto_(prepend|append)_file' /etc/php /var/www --include='*.ini' --include='.htaccess' --include
='.user.ini' 2>/dev/null
# Intégrité des fichiers des paquets PHP/Apache
dpkg --verify $(dpkg-query -W -f='${binary:Package} ' 'php*' 'libapache2-mod-php*' apache2 2>/dev/null)
# Persistance
crontab -l -u www-data 2>/dev/null; ls -la /var/spool/cron/crontabs /etc/cron.d
find /etc/systemd /lib/systemd/system /etc/init.d /etc/ld.so.preload -newermt '2026-10-20' -ls 2>/dev/null
ls -la ~www-data/.ssh 2>/dev/null; getent passwd | awk -F: '$3==0 || $3>=1000'
last -F | head -30; lastlog -t 7
```

F.4 Comptes, sessions, application

Lister les comptes administrateurs applicatifs créés ou modifiés depuis le 20/10 (ex. WordPress : `wp user list --role=administrator --fields=ID,user_login,user_registered` en lecture seule), les tâches planifiées applicatives, les extensions installées récemment.

F.5 Limites

- Une exploitation réussie peut tenir en une requête sans erreur visible ; l'absence d'alerte ne prouve rien.
- Les logs locaux peuvent être modifiés par un attaquant disposant des droits du serveur web ou plus : privilégier une copie centralisée (syslog distant, CloudWatch).
- Les crashs et 500 ont de nombreuses causes bénignes : un indicateur isolé ne prouve pas une exploitation ; corréler horodatage, IP source et PID.

F.6 Si une compromission est plausible

1. **Préserver** avant toute modification : snapshot disque (ou image forensique), capture mémoire avec l'outil validé par votre CSIRT, copie des journaux avec empreintes (`sha256sum`), liste des processus et connexions (sorties F.2).
2. **Isoler** : retirer du load balancer, filtrer entrée/sortie sauf accès d'investigation ; ne pas éteindre avant la capture mémoire si elle est requise.
3. **Faire tourner les secrets** accessibles au compte web : identifiants base de données, clés API, sels/clefs de session applicatifs (invalide les sessions), clés SSH, identifiants cloud (rôle d'instance : révoquer les sessions actives).
4. **Rechercher la persistance** (F.3) sur l'hôte et sur les systèmes joignables avec les secrets exposés.
5. **Reconstruire** le serveur depuis une image saine, PHP corrigé, données restaurées et auditées, si l'un des cas suivants est vrai : exécution de commandes confirmée ; fichiers système ou appartenant à root modifiés ; persistance trouvée ; chronologie impossible à établir ; indice d'élévation de privilèges. **Mettre à jour PHP ne nettoie pas un serveur compromis.**
6. **Obligations** : violation de données personnelles → notification CNIL sous 72 h (RGPD art. 33) si un risque pour les personnes existe ; déclaration possible au CERT-FR.

G. Recommandations AWS (synthèse)

Détail, prérequis, configurations et limites : `AWS_SECURITY.md`.

- AWS indique que ses services ne sont pas affectés ; le PHP exécuté par le client (EC2, conteneurs, Lightsail, plateformes Elastic Beanstalk non mises à jour) relève du client [S10].
- Amazon Linux 2023 : php8.2/8.3/8.4/8.5 corrigés (ALAS2023-2026-3301 à 3304) ; **php8.1 AL2023 et php du cœur AL2 : « No Fix Planned »** [S11].
- Elastic Beanstalk : plateformes PHP v4.13.12 (mitigations) le 21/10, correctif upstream annoncé pour le 22/10 ; Lightsail : nouvelles instances depuis les blueprints mis à jour ; Bitnami non mis à jour par `apt` [S10][S24].
- WAF : règles AWS = contrôles compensatoires ; limite d'inspection du corps (8 Ko ALB) [S13][S23].
- Aucun service AWS n'est documenté comme détectant spécifiquement ces CVE ; GuardDuty Runtime Monitoring et CloudTrail aident à détecter la post-exploitation et l'abus d'identifiants.

H. Plan de validation (checklist)

Avant correctif

- ☐ Inventaire complet produit (`phpaudit.py audit`) et archivé (JSON + Markdown).
- ☐ Pour chaque paquet source PHP : version corrigée de référence identifiée dans la matrice C.2, **source officielle** notée.
- ☐ Version candidate APT ≥ version corrigée (étape S2 du plan généré).
- ☐ Sauvegarde configurations + liste des paquets + snapshot.
- ☐ Plan validé en préproduction (même OS, même branche, mêmes extensions).
- ☐ Fenêtre et retrait du load balancer planifiés ; parties prenantes informées.

Après correctif

- ☐ `PKG` : tous les paquets source PHP en `FIXED` (ou `LIKELY_FIXED` documenté par l'avis distribution).
- ☐ `BIN` : aucun binaire hors paquet vulnérable ou indéterminé.
- ☐ `RUN` : aucun processus avec bibliothèque/binaire PHP `(deleted)` ; `needrestart -b -r l` sans service à redémarrer.
- ☐ Services actifs (`systemctl is-active`), `php-fpmX.Y -t` , `apache2ctl configtest` , `nginx -t` OK.
- ☐ Tests fonctionnels : pages clés en 200, connexion/déconnexion (sessions), formulaire POST, **upload de fichier** de taille nominale et proche de la limite, tâches planifiées.
- ☐ Absence de régression : taux de 5xx et latence comparés à J-1 ; erreurs PHP nouvelles absentes.
- ☐ Conteneurs : images reconstruites, digest déployé = digest corrigé, `php -v` dans les conteneurs en cours.
- ☐ Mitigations temporaires réévaluées (laisser `session.upload_progress.enabled=0` si inutilisé ; retirer `memory_limit=-1` et blocages WAF bloquants une fois corrigé).
- ☐ Recherche de compromission F.1–F.4 exécutée et résultat consigné.

Surveillance post-déploiement : relancer `phpaudit.py audit` à J+1 et J+3 (le contrôle `LOG` compare les indicateurs), alarmes sur DR-005/DR-006/DR-007.

Critères de clôture : verdict `OK` de l'audit sur tous les hôtes du périmètre ; aucun constat `ALERT` non expliqué ; preuves (rapports, versions, sources) archivées ; serveurs EOL soit migrés, soit isolés avec date de migration validée.

I. Stratégie de maintenance

L'outil `php-vuln-audit` (voir `README.md`) sépare données, contrôles, remédiation et rapports. Une nouvelle faille PHP se traite en ajoutant un fichier `data/advisories/<id>.json` (validé par schéma, sources datées), sans modifier le code. Mode audit uniquement ; la remédiation est un script généré, à relire et lancer manuellement.

Sources (consultées le 22/10/2026)

Primaires :

- [S1] PHP Group — Security advisory: Critical remote code execution vulnerabilities, update now, 21/10/2026
- [S2] PHP Group — GHSA-w8hw-w9fx-j79w (CVE-2026-109618)
- [S3] PHP Group — GHSA-6wmf-4rv5-p3rp (CVE-2026-109619)
- [S4] CERT-FR — CERTFR-2026-AVI-1359
- [S5] CERT-FR — CERTFR-2026-ALE-014
- [S6] CERT-EU — Security Advisory 2026-018
- [S7] Debian — security tracker CVE-2026-109618, CVE-2026-109619
- [S8] Canonical — USN-8948-1
- [S9] Canonical — USN-8948-2
- [S10] AWS — Security Bulletin 2026-143-AWS
- [S11] AWS — Amazon Linux Security Center CVE-2026-109618
- [S12] PHP Group — Supported Versions
- [S13] AWS — AWS WAF : inspection du corps des requêtes
- [NCSC] NCSC-NL — php2shell

Secondaires (utilisées uniquement là où elles sont citées) :

- [S14] watchTowr — PHP RCE FAQ (page partiellement incohérente)
- [S15] Check Point — php2shell: What We Know
- [S16] idem S15 (CMS non correctifs)
- [S17] Tanium — PHP2Shell: Responding to CVE-2026-109618 and CVE-2026-109619
- [S18] Wiz — Exploitation in the Wild of wp2shell
- [S19] ma.ttias.be — How to apply the patches (rédigé avant publication des correctifs)
- [S20] LinuxCompatible — Sury.org ships updates
- [S21] Launchpad — build php8.4 8.4.27 (PPA ondrej)
- [S22] Docker (DEV Community) — How to Fix PHP2Shell
- [S23] Classmethod — Test des règles AWS WAF PHP2Shell
- [S24] AWS — Update PHP on Lightsail instances

PARTIE II

Plan de remédiation Debian / Ubuntu

Source : PLAN_REMEDIATION.md

Version 1.0 — 22/10/2026. Référence : `NOTE_CORRECTIF_PHP2SHELL.md` (matrice C.2 pour les versions corrigées).

Règles d'exécution

- Préproduction d'abord, production ensuite, serveur par serveur derrière le load balancer.
- Toutes les commandes de **lecture** (étapes 1, 2, 6, 7) sont sans effet sur le système. Les étapes **3 à 5** modifient le système : elles exigent une validation explicite (go/no-go) du responsable.
- Le script généré par `python3 phpaudit.py plan` reprend les étapes 3 à 6 avec confirmation à chaque étape ; il refuse de démarrer sans `--i-have-reviewed`.
- Ne jamais lancer d'exploit ou de PoC sur un système réel pour « vérifier ».

Étape 0 — Préparation (lecture seule)

```
cd /opt/php-vuln-audit            # emplacement d'installation de l'outil
sudo python3 phpaudit.py audit    # root recommandé pour /proc et les journaux ; aucune écriture hors report
s/
sudo python3 phpaudit.py plan     # génère reports/plan_<hôte>_<env>.md et .sh
```

Attendu : verdict `VULNERABLE`, `ALERT`, `INCOMPLETE` ou `OK` ; code retour 1/2/3/0. Lire en priorité le tableau « Constats prioritaires ».

Étape 1 — Inventaire des versions installées

Commande	Objectif	Résultat attendu
`dpkg-query -W -f='${binary:Package}\t${Version}\t${source:Package}\n' 'php*' 'libapache2-mod-php*' | sort`	Paquets PHP et paquet source	Une ou plusieurs branches `phpX.Y` avec version complète
`apt-cache policy php8.3-common` *(adapter la branche)*	Dépôt d'origine (distribution, `security`, Sury/PPA ondrej)	Ligne `***` pointant le dépôt de la version installée
`ls -1 /usr/bin/php[0-9]* /usr/sbin/php-fpm* 2>/dev/null; update-alternatives --display php`	Binaires et CLI par défaut	Liste des binaires ; `php` → branche par défaut
`find / -xdev \( -name php -o -name 'php-fpm*' -o -name 'libphp*.so' -o -name frankenphp \) -type f 2>/dev/null | xargs -r dpkg -S 2>&1 | grep 'no path found'`	Binaires **hors paquets** (compilés, `/opt/bitnami`, FrankenPHP)	Vide si tout vient de dpkg ; sinon traiter §Cas particuliers
`systemctl list-units --type=service --all 'php*-fpm*' 'apache2*' 'nginx*' --no-pager`	Services concernés	Services actifs et inactifs
`apache2ctl -M 2>/dev/null | grep -Ei 'php|mpm'`	mod_php actif ? MPM ?	`php_module` = exposition maximale

Étape 2 — Version réellement utilisée par chaque site

```
# Apache : mod_php ou proxy FPM par vhost
grep -RniE 'SetHandler|proxy:unix|ProxyPassMatch|AddHandler' /etc/apache2/sites-enabled/ /etc/apache2/conf-e
nabled/
# Nginx : socket FastCGI par site
grep -RnE 'fastcgi_pass' /etc/nginx/sites-enabled/ /etc/nginx/conf.d/ 2>/dev/null
# Socket -> pool -> branche
grep -RnE '^\s*(\[|listen\s*=|user\s*=)' /etc/php/*/fpm/pool.d/
# Masters FPM en cours et fichier de conf (révèle la branche)
ps -eo pid,user,cmd | grep -E 'php-fpm: master'
```

Attendu : table vhost → handler → socket → pool → branche. `php -v` n'indique que le CLI.

Optionnel (préproduction uniquement, à supprimer immédiatement) : un fichier `<?php echo PHP_VERSION, ' ', PHP_SAPI, ' ', php_ini_loaded_file();` déposé puis supprimé, appelé via `curl -s` en local. En production, préférer le croisement ci-dessus. `X-Powered-By` est non fiable (rétroportages, `expose_php=Off`).

Étape 3 — Sauvegarde et compatibilité *(modifie le système : go/no-go)*

```
BK=/root/php-remediation-$(date -u +%Y%m%dT%H%M%SZ); mkdir -p "$BK"
tar czf "$BK/etc.tgz" /etc/php /etc/apache2 /etc/nginx 2>/dev/null
dpkg-query -W -f='${binary:Package}\t${Version}\n' 'php*' 'libapache2-mod-php*' > "$BK/packages-before.tsv"
for v in /usr/bin/php[0-9]*; do "$v" -m > "$BK/modules-$(basename "$v").txt"; done
cp -a /var/cache/apt/archives/php*.deb "$BK/" 2>/dev/null || true
```

- Snapshot machine/volume si disponible (point de retour le plus sûr).
- **Compatibilité** : un correctif de sécurité dans la même branche (ex. 8.3.6-…12 → …13, ou 8.4.26 → 8.4.27) ne change pas l'API ; le risque applicatif est faible mais doit être validé en préproduction. **Ne pas changer de branche** (8.1 → 8.4) dans l'urgence sans campagne de tests dédiée.

Étape 4 — Mise à jour depuis une source confirmée *(go/no-go)*

1. Rafraîchir les index : `sudo apt-get update`
2. **Vérifier la version candidate avant d'installer** :

   ```
   apt-cache policy php8.3-fpm | sed -n '1,4p'
   dpkg --compare-versions "$(apt-cache policy php8.3-fpm | awk '/Candidate:/{print $2}')" ge 8.3.6-0ubuntu
   0.24.04.13 && echo "CANDIDATE CORRIGÉE" || echo "STOP : pas de correctif disponible"
   ```

   Remplacer le paquet et la version de référence selon la matrice C.2 (Debian 13 : `8.4.27-1~deb13u1` ; Debian 12 : `8.2.35-1~deb12u1` ; dépôt Sury : version upstream `8.x.y` corrigée).
3. Preuves complémentaires de rétroportage :
   - Ubuntu : `pro fix CVE-2026-109618` puis `pro fix CVE-2026-109619` (indique si le correctif est installé, disponible, ou requiert Ubuntu Pro).
   - Debian : `debsecan --suite "$(. /etc/os-release; echo $VERSION_CODENAME)" | grep -E 'CVE-2026-10961[89]'` (paquet `debsecan`).
   - Changelog **après** installation : `zgrep -E 'CVE-2026-10961[89]' /usr/share/doc/php8.3-common/changelog.Debian.gz`.

4. Si STOP : ne pas installer de version arbitraire (dépôt inconnu, `.deb` téléchargé à la main). Appliquer §E de la note (mitigations + isolement) et suivre l'avis de la distribution.
5. Installer **uniquement** les paquets PHP du cœur déjà présents :

   ```
   sudo apt-get install --only-upgrade $(dpkg-query -W -f='${binary:Package} ${source:Package} ${db:Status-A
   bbrev}\n' 'php*' 'libapache2-mod-php*' | awk '$2 ~ /^php[0-9]/ && $3 ~ /^ii/ {print $1}')
   ```

   Pas d'`apt upgrade` global dans la fenêtre : il élargit le périmètre de changement sans améliorer la correction.

Étape 5 — Redémarrage contrôlé *(go/no-go)*

Ordre : retirer le serveur du load balancer → redémarrer → tests → réintégrer.

```
sudo php-fpm8.3 -t && sudo systemctl restart php8.3-fpm && systemctl is-active php8.3-fpm
sudo apache2ctl configtest && sudo systemctl restart apache2 && systemctl is-active apache2
```

- mod_php : `restart` (stop/start) obligatoire ; `reload` / `apachectl graceful` ne rechargent pas `libphp` de façon fiable [S19 de la note].
- Redémarrer aussi les workers CLI longue durée (supervisord, systemd, `php artisan queue:restart`).
- LiteSpeed : relancer les processus `lsphp`.

Étape 6 — Vérification (lecture seule)

```
dpkg-query -W -f='${binary:Package}\t${Version}\n' 'php*' 'libapache2-mod-php*'
for d in /usr/share/doc/php*/changelog.Debian.gz; do printf '%s: ' "$d"; zgrep -cE 'CVE-2026-10961[89]'
"$d"; done
grep -lE '(php|libphp|apache2).*\(deleted\)$' /proc/[0-9]*/maps 2>/dev/null || echo "Aucun processus obsolèt
e"
command -v needrestart >/dev/null && sudo needrestart -b -r l
sudo python3 phpaudit.py audit
```

Attendu : versions ≥ référence, compteurs CVE > 0 sur les paquets `php8.x-*` du cœur, aucun processus obsolète, verdict `OK` (ou constats résiduels documentés).

Tests fonctionnels : pages clés (200), authentification et persistance de session, POST de formulaire, upload nominal et proche de la limite, tâches cron applicatives, comparaison du taux de 5xx avec la veille.

Étape 7 — Contrôle post-déploiement et retour arrière

- J+1 / J+3 : relancer l'audit ; surveiller DR-003/005/006/007 ; vérifier l'absence de nouveaux `.php` dans les webroots (§F.3 de la note).
- Remettre en état les mitigations temporaires devenues inutiles (`memory_limit=-1`, blocage multipart WAF) ; conserver `session.upload_progress.enabled=0` si la fonction n'est pas utilisée.

Retour arrière (réintroduit la vulnérabilité : serveur **isolé** uniquement)

1. Configuration : `tar xzf "$BK/etc.tgz" -C /` puis redémarrage des services.
2. Paquets : `dpkg -i "$BK"/php*.deb` si les anciens `.deb` étaient en cache ; sinon `apt-get install paquet=version` si la version est encore publiée (souvent retirée de la poche sécurité).
3. Option la plus fiable : restauration du snapshot pris à l'étape 3.

Cas particuliers

Cas	Procédure
Branche EOL (≤ 8.1) Debian/Ubuntu	Ubuntu 22.04 (php8.1) et 20.04/18.04 (Pro) disposent d'un backport (matrice C.2). Ailleurs : backport vendeur vérifié ou migration ; en attendant, isolement + mitigations
Dépôt Sury, branche EOL	Lire `/usr/share/doc/<paquet>/changelog.Debian.gz` après mise à jour ; si aucune CVE citée, considérer vulnérable
Binaire compilé (`/usr/local`)	Recompiler depuis 8.2.35 / 8.3.36 / 8.4.27 / 8.5.12 (sources php.net), ou depuis les branches de backport 7.4/8.0/8.1 publiées par l'équipe PHP ; redémarrer
Bitnami (`/opt/bitnami/php`)	Non mis à jour par `apt` ; migrer vers une pile gérée par paquets ou mettre à jour la pile Bitnami (procédure AWS Lightsail)
Conteneurs	`FROM php:8.x.y-fpm` corrigé (ou digest), `docker compose build --pull --no-cache` , `docker compose up -d --force-recreate` , vérifier `php -v` dans le conteneur ; `php:8.2-fpm` flottant pointait encore 8.2.34 le 21/10 (source secondaire)
Panneaux (cPanel/Plesk)	Suivre l'avis de l'éditeur du panneau (PHP fourni hors dépôt distribution) — **À confirmer** par éditeur

PARTIE III

Recommandations AWS

Source : AWS_SECURITY.md

Version 1.0 — 22/10/2026. Complète `NOTE_CORRECTIF_PHP2SHELL.md`. Commandes AWS CLI données en **lecture seule** sauf mention « modifie ».

1. Partage des responsabilités

Périmètre	Responsable	État au 22/10/2026
Services AWS eux-mêmes	AWS	« AWS services are not impacted by these issues » [A1]
Amazon WorkMail	AWS	Contrôles de configuration appliqués ; montée en 8.3.36 visée le 22/10 ; aucune action client [A1]
Elastic Beanstalk (plateforme PHP)	AWS publie, **le client applique**	v4.13.12 (21/10) avec mesures provisoires ; version avec correctif upstream annoncée pour le 22/10 ; mise à jour auto seulement si *Managed Platform Updates* activé [A1]
Amazon Linux (paquets)	AWS publie, **le client installe**	Voir §2 [A2]
Lightsail	**Client**	Blueprints mis à jour ; instances existantes non modifiées : recréer ou mettre à jour [A1][A3]
EC2 Debian/Ubuntu, conteneurs ECS/EKS, PHP compilé, Lambda custom	**Client**	Procédure `PLAN_REMEDIATION.md` / §6

2. EC2 — mise à jour du système et du moteur PHP

Amazon Linux [A2]

Version	Paquet	Statut	Avis
AL2023	php8.2 / php8.3 / php8.4 / php8.5	Fixed	ALAS2023-2026-3301 / 3302 / 3303 / 3304
AL2023	php8.1	**No Fix Planned**	—
AL2 Extras	php8.2	Fixed	ALAS2PHP8.2-2026-015
AL2 Core	php	**No Fix Planned**	—

Versions de paquet corrigées : non affichées par l'ALAS Explorer — **À confirmer** dans chaque avis ALAS.

```
# AL2023 (lecture)
dnf updateinfo list --security 2>/dev/null | grep -i php
rpm -qa 'php*' | sort
# AL2023 (modifie) — après go/no-go
sudo dnf upgrade --advisory=ALAS2023-2026-3302    # adapter à la branche installée
```

Risque traité : 618 et 619 sur l'hôte. **Limites** : php8.1 AL2023 et php AL2 cœur ne seront pas corrigés → migration vers php8.2+ (AL2023) ou isolement.

Debian/Ubuntu sur EC2 : `PLAN_REMEDIATION.md` à l'identique. Redémarrer PHP-FPM/Apache après mise à jour.

3. AWS Systems Manager — inventaire et correctifs

Fonction	Usage ici	Prérequis	Limites
Inventory	Lister les paquets PHP de toute la flotte	Agent SSM, rôle d'instance avec `AmazonSSMManagedInstanceCore`, association Inventory	Ne voit pas les binaires compilés ni le PHP des conteneurs
Patch Manager — Scan	État de conformité des correctifs	Patch baseline adaptée	Les baselines Ubuntu/Debian approuvent par priorité/section ; la documentation consultée ne décrit pas de sélection par CVE pour ces OS [A4] → compléter par l'audit ci-dessous
Patch Manager — Install	Installation (modifie)	Fenêtre de maintenance	`RebootIfNeeded` par défaut : redémarrage machine (élimine les processus obsolètes, mais impact prod) ; `NoReboot` laisse l'ancien PHP en mémoire → redémarrer FPM/Apache séparément
Run Command	Exécuter `phpaudit.py audit` en lecture seule sur la flotte	Outil déployé sur l'instance, bucket S3 de sortie	Résultats dans S3/console ; aucune action corrective

```
# Paquets PHP installés sur une instance (lecture)
aws ssm list-inventory-entries --instance-id i-0123456789abcdef0 --type-name AWS:Application \
  --filters Key=Name,Values=php,Type=BeginWith --query 'Entries[].[Name,Version]' --output table
# Scan de conformité (n'installe rien)
aws ssm send-command --document-name AWS-RunPatchBaseline --parameters Operation=Scan \
  --targets Key=tag:Role,Values=web
# Audit PHP2Shell en lecture seule sur la flotte
aws ssm send-command --document-name AWS-RunShellScript --targets Key=tag:Role,Values=web \
  --parameters '{"commands":["cd /opt/php-vuln-audit && python3 phpaudit.py audit --output-dir /var/tmp/phpa
udit; echo exit=$?"]}' \
  --output-s3-bucket-name <bucket-audit> --output-s3-key-prefix php2shell/
```

4. AWS WAF (ALB, CloudFront, API Gateway, App Runner…)

Ce que publie AWS [A1] :

- CVE-2026-109618 : règle `SizeRestrictions_BODY` du groupe géré **Core rule set** (bloque les corps > 8 Ko).
- CVE-2026-109619 : **groupe géré PHP version 2.3**, promu version par défaut, et règle personnalisée `PHP2Shell` (priorité 0, Block) : méthode POST **et** `PHPSESSID` (clé de cookie ou corps) **et** `PHP_SESSION_UPLOAD_PROGRESS` dans le corps ; *OversizeHandling* `CONTINUE`.
- AWS qualifie ces règles de **contrôles compensatoires**, pas de mitigations complètes.

Reconstitution JSON de la règle `PHP2Shell` pour l'éditeur de règles de la console **[Reco — équivalent fonctionnel, à comparer avec le JSON officiel du bulletin avant usage]** :

```
{
  "Name": "PHP2Shell",
  "Priority": 0,
  "Action": { "Count": {} },
  "VisibilityConfig": { "SampledRequestsEnabled": true, "CloudWatchMetricsEnabled": true, "MetricName": "PHP
2Shell" },
  "Statement": { "AndStatement": { "Statements": [
    { "ByteMatchStatement": { "FieldToMatch": { "Method": {} }, "PositionalConstraint": "EXACTLY", "SearchSt
ring": "POST",
        "TextTransformations": [ { "Priority": 0, "Type": "NONE" } ] } },
    { "OrStatement": { "Statements": [
      { "ByteMatchStatement": { "FieldToMatch": { "SingleHeader": { "Name": "cookie" } }, "PositionalConstra
int": "CONTAINS", "SearchString": "PHPSESSID=",
          "TextTransformations": [ { "Priority": 0, "Type": "NONE" } ] } },
      { "ByteMatchStatement": { "FieldToMatch": { "Body": { "OversizeHandling": "CONTINUE" } }, "PositionalC
onstraint": "CONTAINS", "SearchString": "PHPSESSID",
          "TextTransformations": [ { "Priority": 0, "Type": "NONE" } ] } } ] } },
    { "ByteMatchStatement": { "FieldToMatch": { "Body": { "OversizeHandling": "CONTINUE" } }, "PositionalCon
straint": "CONTAINS", "SearchString": "PHP_SESSION_UPLOAD_PROGRESS",
        "TextTransformations": [ { "Priority": 0, "Type": "NONE" } ] } } ] } }
}
```

Règle de taille (alternative documentée par un test indépendant) : `SizeConstraintStatement` sur `Body`, `OversizeHandling: MATCH`, `ComparisonOperator: GT`, `Size: 8192` [A6].

Limites vérifiées

- Inspection du corps : **8 Ko fixes** pour ALB et AppSync ; 16 Ko par défaut, jusqu'à 64 Ko, pour CloudFront, API Gateway, Cognito, App Runner, Verified Access [A5].
- Avec `CONTINUE`, seuls les premiers octets inspectables sont évalués : le PoC public place les marqueurs vers 56 Ko et passe la règle `PHP2Shell` sur ALB ; seule la règle de taille l'a bloqué lors du test [A6].
- La règle de taille bloque aussi tout upload, JSON ou PUT/PATCH > 8 Ko : faux positifs massifs sur les applications avec upload → **mode Count d'abord**, puis Block ciblé par chemin.
- Contournements : `session.name` ≠ `PHPSESSID` ou `session.upload_progress.name` personnalisé ; CVE-2026-109618 n'a pas besoin de ces chaînes.
- Le WAF ne protège que le trafic passant par la ressource associée : restreindre le Security Group des instances au seul ALB, ou à la *prefix list* gérée CloudFront `com.amazonaws.global.cloudfront.origin-facing` pour une origine CloudFront.

5. Journalisation et investigation

Service	Ce qu'il voit réellement	Ce qu'il ne voit pas
ALB access logs (S3)	Ligne de requête, statut ELB/cible, tailles reçue/envoyée, IP client	Corps, `Content-Type`
AWS WAF logs (CloudWatch Logs / S3 / Firehose)	En-têtes, règle déclenchée, action	Corps de la requête
CloudWatch Logs (agent)	Ce que vous y envoyez : logs Apache/Nginx/FPM/kern	Rien de plus que les fichiers source
CloudTrail	Appels d'API AWS (y compris avec les identifiants du rôle d'instance)	Aucune requête HTTP applicative ; pas l'exploitation elle-même

Service	Ce qu'il voit réellement	Ce qu'il ne voit pas
VPC Flow Logs	Flux réseau (connexions sortantes inhabituelles)	Contenu

```
# Filtre de métrique sur les workers tués (modifie : crée un filtre)
aws logs put-metric-filter --log-group-name /web/php-fpm --filter-name php-worker-signal \
  --filter-pattern '"exited on signal"' \
  --metric-transformations metricName=PhpWorkerSignal,metricNamespace=Web/PHP,metricValue=1
```

CloudWatch Logs Insights (lecture) :

```
fields @timestamp, @logStream, @message
| filter @message like /exited on signal (6|7|11)|AH00052|Allowed memory size of/
| stats count() by bin(10m), @logStream
```

CloudTrail — usage des identifiants d'une instance web depuis le 20/10 (la session du rôle d'instance porte l'ID d'instance) :

```
aws cloudtrail lookup-events --lookup-attributes AttributeKey=Username,AttributeValue=i-0123456789abcdef0 \
  --start-time 2026-10-20T00:00:00Z --query 'Events[].[EventTime,EventName,EventSource]' --output table
```

6. Détection gérée : GuardDuty, Inspector, Security Hub

- **Aucune source AWS consultée n'affirme une détection spécifique de CVE-2026-109618/109619.** Ne pas présumer de couverture.
- **GuardDuty** : `UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.OutsideAWS` signale l'usage des identifiants d'une instance hors AWS (vol après RCE). **Runtime Monitoring** (agent ; EC2, ECS sur Fargate, EKS hors Fargate) observe exécutions de processus, accès fichiers et connexions réseau [A7] : pertinent pour la post-exploitation (shell lancé par le worker web). Noms exacts des findings runtime : à vérifier dans la référence GuardDuty.
- **Amazon Inspector** : scanne les paquets EC2 (via SSM) et images ECR ; détection de ces CVE dépendante de la mise à jour de sa base — vérifier en filtrant par `vulnerabilityId`. Le PHP compilé hors paquet n'est pas couvert.
- **Security Hub** : agrège GuardDuty/Inspector ; utile pour le suivi de clôture, pas comme détecteur.

7. Conteneurs (ECS, EKS, ECR)

1. Mettre à jour la base : `FROM php:8.4.27-fpm` (ou digest épinglé) ; ne pas compter sur `php:8.2-fpm` flottant (pointait encore 8.2.34 le 21/10 selon une source secondaire [A8]).
2. Reconstruire avec `--pull --no-cache`, pousser dans ECR, laisser Inspector scanner l'image.
3. Redéployer (modifie) :

   ```
   aws ecs update-service --cluster <c> --service <s> --force-new-deployment
   kubectl rollout restart deployment/<d> -n <ns>
   ```

4. Vérifier les **tâches/pods en cours** (lecture) :

```
aws ecs describe-tasks --cluster <c> --tasks $(aws ecs list-tasks --cluster <c> --service-name <s> --quer
y 'taskArns[]' --output text) \
 --query 'tasks[].containers[].[name,imageDigest]' --output table
kubectl get pods -n <ns> -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{..imageID}{"\n"}{end}'
kubectl exec -n <ns> <pod> -- php -v
```

Une image corrigée dans ECR ne protège aucun conteneur déjà lancé.

8. Lambda et environnements PHP personnalisés

Lambda n'a pas de runtime PHP géré ; PHP n'y existe que via runtime personnalisé (`provided.al2023`) ou image conteneur (ex. couches communautaires). Si c'est votre cas : reconstruire la couche/l'image avec PHP corrigé et publier une nouvelle version de fonction. Exposition exacte (réutilisation d'environnement d'exécution, FPM embarqué ou non) : **À confirmer** auprès du fournisseur de la couche.

9. IAM, secrets, réseau — réduction du rayon d'impact

Mesure	Risque traité	Configuration à vérifier	Limite
IMDSv2 obligatoire	SSRF vers les métadonnées	`aws ec2 modify-instance-metadata-options --instance-id <i> --http-tokens required --http-put-response-hop-limit 1` (modifie)	Une RCE locale peut toujours appeler IMDSv2
Rôle d'instance minimal	Abus post-RCE	Pas de `*:*`, pas d'accès S3 global ; Access Analyzer	—
Secrets Manager / Parameter Store	Secrets en clair dans `.env` / `wp-config.php`	Rotation prête à déclencher	Le processus PHP lit le secret : rotation obligatoire après compromission
Security Groups	Exposition directe	Entrée 80/443 uniquement depuis ALB/CloudFront ; **aucun port 9000 (FastCGI)**	—
Filtrage sortant (SG/NACL/Network Firewall)	Téléchargement d'outils, shell inversé	Sortie limitée (dépôts, API nécessaires)	Mise en place progressive
Comptes séparés (prod/préprod)	Propagation	AWS Organizations, SCP	—

En cas de suspicion de compromission (modifie) : basculer l'instance en *standby* de l'Auto Scaling group, lui appliquer un Security Group d'isolement, créer un snapshot EBS (`aws ec2 create-snapshot --volume-id <vol> --description "IR php2shell"`), révoquer les sessions actives du rôle d'instance (console IAM « Revoke active sessions »), faire tourner les secrets, puis reconstruire depuis une AMI saine (cf. note §F.6).

Sources (consultées le 22/10/2026)

- [A1] AWS — Security Bulletin 2026-143-AWS
- [A2] AWS — Amazon Linux Security Center CVE-2026-109618
- [A3] AWS — Update PHP on Lightsail instances
- [A4] AWS — Patch Manager : baselines prédéfinies et personnalisées
- [A5] AWS — AWS WAF : inspection du corps

- [A6] Classmethod (secondaire) — Test des règles AWS WAF PHP2Shell
- [A7] AWS — GuardDuty Runtime Monitoring
- [A8] Docker / DEV Community (secondaire) — How to Fix PHP2Shell

PARTIE IV

Outil php-vuln-audit : utilisation et maintenance

Source : README.md

Audit **en lecture seule** des vulnérabilités du moteur PHP sur Debian/Ubuntu, piloté par des fichiers de données. Premier avis embarqué : PHP2Shell (CVE-2026-109618, CVE-2026-109619).

Principes

- **Mode audit uniquement** : le schéma de configuration n'accepte que `"mode": "audit"`. L'outil exécute des commandes de lecture (`dpkg-query`, `dpkg -S`, `<binaire> -v`, `systemctl list-units`, lecture de fichiers et de `/proc`) et n'écrit que dans `reports/` et `data/inventory/`.
- **Détection ≠ recommandation ≠ application** : `checks/` constate, `core/report.py` recommande, `remediation/` génère un script **à relire**, exécuté manuellement, protégé par `--i-have-reviewed` et une confirmation par étape.
- **Données, pas code** : versions corrigées, correctifs de distribution, mitigations, règles de détection et sources (datées) vivent dans `data/` et `config/`, validés par schéma.
- **Bibliothèque standard Python 3** uniquement (testé avec Python 3.13) ; `jsonschema` utilisé s'il est présent, sinon validateur intégré.

Installation

```
sudo git clone <dépôt> /opt/php-vuln-audit     # ou copie de l'archive
cd /opt/php-vuln-audit
python3 phpaudit.py validate                    # configuration, avis et règles valides
python3 -m unittest discover -s tests           # 49 tests
```

Utilisation

```
sudo python3 phpaudit.py audit                          # collecte + évaluation + rapports MD/JSON
sudo python3 phpaudit.py audit --skip-logs              # sans analyse des journaux
sudo python3 phpaudit.py audit --checks PKG RUN         # sous-ensemble de contrôles
sudo python3 phpaudit.py collect -o inv.json            # inventaire seul (à transférer)
python3 phpaudit.py audit --inventory inv.json          # analyse hors ligne d'un inventaire
python3 phpaudit.py plan --inventory inv.json           # plan de remédiation (.md + .sh à relire)
python3 phpaudit.py --env preproduction audit           # profil d'environnement
python3 phpaudit.py list-checks
```

`root` est recommandé (lecture de `/proc/*/maps` et des journaux) ; en non-root, le rapport signale les données manquantes.

Codes retour : `0` OK · `1` VULNERABLE · `2` ALERT (indicateurs dans les journaux) · `3` INCOMPLETE (statuts indéterminés) · `4` erreur de configuration. Utilisables en CI ou via SSM Run Command.

Statuts : `VULNERABLE`, `ALERT`, `UNKNOWN` (à confirmer), `WARN` (mitigation/exposition), `LIKELY_FIXED` (backport prouvé par le changelog mais absent de la table), `FIXED`, `INFO`.

Contrôles

ID	Module	Vérifie
PKG	`checks/packages.py`	Paquets source `phpX.Y` : table de correctifs distribution (comparaison de versions Debian), sinon version upstream, sinon preuve de backport dans `changelog.Debian.gz` ; branches EOL
BIN	`checks/binaries.py`	Binaires PHP hors dpkg (compilés, `/opt/*`, embarqués) : `apt` ne les corrige pas
RUN	`checks/runtime.py`	Processus exécutant un PHP supprimé (mise à jour sans redémarrage), services actifs, mod_php
MIT	`checks/mitigations.py`	Valeur effective des directives de mitigation par branche/SAPI/pool FPM (php_admin > php_value > php.ini > défaut), surcharges `.htaccess`
EXT	`checks/external.py`	Conteneurs (option) et en-têtes `X-Powered-By` (sondes HTTP optionnelles)
LOG	`checks/logs.py`	Règles regex de `config/detection_rules.json` sur les journaux configurés (gz pris en charge, secrets masqués)

Architecture

```
config/       settings.json (+ schéma) · detection_rules.json (+ schéma) — seuils, chemins, environnements
data/         advisories/<id>.json (+ schéma) — CVE, versions corrigées, correctifs distro, mitigations, sou
rces datées
              inventory/ — inventaires collectés
checks/       contrôles modulaires (registre dans checks/__init__.py)
remediation/  planner.py + platforms/<plateforme>.json — étapes, commandes, retour arrière
core/         collecte, versions Debian, validation de schéma, moteur, rapports
reports/      rapports générés (Markdown + JSON), plans
tests/        tests unitaires et fixtures (inventaires, journaux)
```

Configuration

`config/settings.json` (validé par `config/settings.schema.json`, clés inconnues refusées) :

- `collect.php_binary_globs` — où chercher des binaires PHP (ajouter vos chemins d'installation).
- `collect.web_roots` — racines scannées pour les `.htaccess` contenant `php_value` / `php_flag`.
- `collect.containers.enabled` / `exec_version_probe` — inventaire Docker (désactivé par défaut ; la sonde exécute `php -r` dans le conteneur).
- `collect.http_probes` — URL interrogées en `HEAD` pour `X-Powered-By`.
- `detection.log_sources` — journaux par type (`access`, `error`, `fpm`, `system`, `waf`) ; `min_hits_to_flag` — seuils par sévérité ; `max_file_age_days`, `max_lines_per_file`, `max_samples_per_rule`.
- `environments` — profils (production, préproduction, lab) ; `allow_remediation_script` contrôle la génération du `.sh`.

Maintenance

Ajouter une nouvelle vulnérabilité PHP

1. Copier `data/advisories/php2shell-2026-10.json` vers `data/advisories/<id>.json`.
2. Renseigner : CVE, `upstream.branches.<X.Y>.fixed`, `min_supported_branch`, `distro_fixes` (une entrée par distribution/release/paquet source, **avec URL et date de vérification**), `mitigations`, `sources`.
3. Ajouter le chemin dans `settings.json > advisories`.
4. `python3 phpaudit.py validate` puis `python3 -m unittest discover -s tests`.
5. Ajouter des fixtures si la logique de versions diffère ; consigner dans `CHANGELOG.md`.

Mettre à jour un avis existant

- Nouvelle release distribution corrigée → ajouter une entrée `distro_fixes` (ex. Debian sid, ELTS) et mettre à jour `verified_on`.
- Écart signalé « À confirmer » (ex. tracker Debian bookworm/619) → vérifier la source, mettre à jour la `note`.
- Ne jamais saisir une version corrigée sans source primaire.

Ajouter une règle de détection

Ajouter un objet dans `config/detection_rules.json` (`id` `DR-NNN`, `applies_to`, `pattern`, `severity`, `rationale`, `false_positives`) ; le chargement refuse regex invalides et doublons. Ajouter un cas positif dans `tests/test_detection.py` (le test échoue si une règle n'en a pas).

Ajouter une plateforme de remédiation

Créer `remediation/platforms/<nom>.json` (`applies_to_os`, `steps`, `rollback`). Les commandes acceptent les variables `{binary_packages}`, `{backup_dirs}`, `{timestamp}` ; `commands_per_service` reçoit `{service}`.

Cadence recommandée

- Hebdomadaire et après chaque avis PHP : `audit` sur la flotte (cron/SSM), archivage des JSON.
- Revue mensuelle des `verified_on` > 30 jours dans `data/advisories`.

Limites connues

- Debian/Ubuntu (dpkg) uniquement pour PKG et le plan ; RPM/Alpine/Windows non couverts.
- Les pools FPM inclus hors `/etc/php/*/fpm/pool.d/` et les directives `php_admin_*` dans les vhosts Apache ne sont pas résolus individuellement (compte global des surcharges uniquement).
- La version servie par un vhost donné est déduite par l'administrateur (`PLAN_REMEDIATION.md` étape 2), pas par l'outil.
- La détection par journaux dépend des formats ; les formats par défaut ne contiennent ni `Content-Type` ni corps. Absence d'alerte ≠ absence de compromission.
- Le contrôle EXT de conteneurs nécessite l'accès au socket Docker.
- L'outil ne prouve pas l'absence de compromission et ne nettoie rien.